arXiv:2405.18047v1 [cs.LG] 28 May 2024

2BP: 2-Stage Backpropagation

Christopher Rae
crae@ed.ac.uk
EPCC, The University of Edinburgh

Joseph Lee
j.lee@epcc.ed.ac.uk
EPCC, The University of Edinburgh

James Richings
j.richings@epcc.ed.ac.uk
EPCC, The University of Edinburgh

Abstract

As Deep Neural Networks (DNNs) grow in size and complexity, they often exceed the memory capacity of a single accelerator, necessitating the sharding of model parameters across multiple accelerators. Pipeline parallelism is a commonly used sharding strategy for training large DNNs. However, current implementations of pipeline parallelism are being unintentionally bottlenecked by the automatic differentiation tools provided by ML frameworks. This paper introduces 2-stage backpropagation (2BP). By splitting the backward propagation step into two separate stages, we can reduce idle compute time. We tested 2BP on various model architectures and pipelining schedules, achieving increases in throughput in all cases. Using 2BP, we were able to achieve a 1.70x increase in throughput compared to traditional methods when training a LLaMa-like transformer with 7 billion parameters across 4 GPUs.

1 Introduction

In the era of big data and artificial intelligence, deep neural networks (DNNs) have emerged as powerful tools for a wide range of applications, from natural language processing to genomic sequencing. As these models grow in size and complexity they can no longer fit within the memory of a single accelerator, leading to a variety of novel techniques being introduced, including 3D parallelism[29, 26, 17], Zero Redundancy Optimizer(ZeRO) [25] and parameter offloading to host memory or NVMe storage [9, 13, 17, 27]. 3D parallelism refers to the three main strategies for parallelising the computation of DNNs: firstly, data parallelism, which refers to the technique of dividing a dataset across multiple accelerators each with their own copies of the model; pipeline parallelism, which distributes model layers across multiple accelerators; and lastly tensor parallelism, which splits individual model layers across multiple accelerators. While data parallelism does not directly address the issue of increasing model sizes, it enhances processing speed and can be used to reduce the memory required for activations by reducing the local batch size for each accelerator. The last two forms of parallelism fall under the umbrella of model parallelism, and are often necessitated by models with sizes exceeding the capacity of a single accelerator. In particular, tensor parallelism is particularly effective only for extremely large models due to large amount communication overhead, which results in inefficient resource utilization when applied to smaller models.

Focusing on pipeline parallelism, naive implementations often result in a long proportion of idle compute time, which is caused by computational dependencies. The ratio of idle compute to compute is often referred to as the *bubble ratio*. To reduce the bubble ratio, a range of pipeline parallelism algorithms have been developed [14, 8, 18, 11, 34, 35, 19]. Pipeline parallel algorithms can be split into 2 categories: synchronous and asynchronous. Synchronous algorithms require a flush at the end of each training step, in order to keep all of the accelerators training on the same version of the model

Preprint. Under review.

weights. Asynchronous algorithms do not preform this flush, often resulting in a smaller bubble ratio at the cost of not converging as well as synchronous algorithms [11, 34, 35].

Deep neural networks are generally composed of a sequence of layers. The output of each layer l is a function of the parameters (e.g. weights and biases) within that layer w_l and the output of the previous layer z_{l-1}, such that $z_l = f_l(z_{l-1}, w_l)$. The basic training of DNNs involves three primary steps: forward propagation, used to calculate the intermediate activations, backward propagation, where we calculate the gradients of the model's parameters and finally, the optimizer step which updates the model's parameters using the calculated gradient. In practice, the backward propagation step is typically abstracted away by the machine learning framework, and not directly programmed by the user. ML frameworks such as PyTorch [2] and TensorFlow [1] use reverse mode automatic differentiation to calculate the gradients, which often involves tracing the computation graph produced by the forward pass at runtime and calculating the derivatives by starting at the end of the graph and propagating to the beginning. The intuition behind reverse mode automatic differentiation stems from the chain rule. As we backward propagate through each layer in our neural network, two calculations take place: the partial derivative of the loss with respect of the parameters $\frac{\partial L}{\partial w_l}$ and the partial derivative of the loss with respect to the input of the layer $\frac{\partial L}{\partial z_{l-1}}$. Both operations depend on a combination of the activations calculated in the forward propagation and the derivative of the loss with respect to the output $\frac{\partial L}{\partial z_l}$. Both of these calculations are handled by the ML framework under a single predefined backward function. This approach works well in a single accelerator setting as it is more memory efficient not having to store the intermediate derivatives $\frac{\partial L}{\partial z_l}$ for later computation, but as we scale our model and pipeline parallelism becomes necessary, the calculation of $\frac{\partial L}{\partial w_l}$ is not immediately required, and can be reordered such that it is calculated only after the backward propagation has started on the preceding accelerator. The objective of this work is to apply this splitting and reordering of the backward propagation step, and measure the performance gained for a variety of model architectures.

2 Background & related work

A wide range of pipeline parallelism schedules have been introduced and studied. The concept of a pipelining schedule was first introduced with GPipe [14], a pipeline parallelism library developed by Google. GPipe introduced the innovative idea of splitting mini-batches further into micro-batches, allowing different accelerators to train on multiple micro-batches in parallel. This approach effectively decreases the bubble ratio by allowing for overlapped compute between accelerators, thereby enhancing the efficiency of the training process.

The next prominent synchronous pipelining schedule to emerge was 1F1B (1 forward, 1 backward), sometimes referred to as PipeDream-Flush. This schedule was originally proposed by PipeDream [11] as an asynchronous pipeline schedule, but was later adapted for use in Megatron-LM [29] and DAPPLE [8] as a synchronous pipelining schedule. Megatron-LM is a research-oriented framework designed for large language model (LLM) training at scale, while DAPPLE is a synchronous training framework that combines data parallelism and pipeline parallelism for training large DNNs. It is also important to note that DeepSpeed [26] independently developed a pipeline scheduling algorithm that is equivalent to 1F1B. To further reduce the bubble ratio when using 1F1B, an interleaved pipelining schedule [29] can be used. We can use this interleaved schedule to decrease the idle compute at the cost of an increase in communication.

Chimera [18] is another noteworthy bidirectional pipeline scheduling algorithm. This method involves storing two copies of the model across the accelerators, which results in doubling the memory consumed by the model's weights. Despite this increase in memory usage and the need for a more complex communication scheme, Chimera significantly reduces the bubble ratio by up to 50%, making it a highly efficient scheduling strategy.

There are a handful of libraries that have implemented tools to utilise pipeline parallelism at runtime. FairScale [7] provides an implementation of GPipe for PyTorch sequential modules. DeepSpeed offers their own implementation of 1F1B also for PyTorch sequential modules. Megatron-LM allows users to use 1F1B or 1F1B interleaved to train LLMs. Colossal-AI [17] (much like Megatron-LM) offers 1F1B or 1F1B interleaved support for a wide range of transformers.

At a similar time as this work is produced, Zero Bubble Pipeline Parallelism [23] was introduced, which also separates the backward propagation step into 2 separate stages. The authors adds support for 2BP (described in later Section 3) on top of the linear layers in Megatron-LM as well as implementing a custom pipelining schedules that uses 2BP to further reduce bubble time to theoretically zero. Our work evaluates the performance of 2BP on a wide range of model architectures and evaluates the scaling performance of the strategy.

3 2-Stage Backpropagation

3.1 Definitions

For each layer during back propagation, we define the computation of the gradient w.r.t to the output of the preceding layer $\frac{\partial L}{\partial z_{l-1}}$ as 'backward-p1' and computation of the gradient w.r.t to the layer parameters $\frac{\partial L}{\partial w_l}$ as 'backward-p2'. We describe 2BP as the process of splitting up the backward pass into backward-p1 and backward-p2, and delaying the computation of backward-p2 in order to maximise accelerator utilization. 2BP can be applied on top of any pipelining schedule, including Gpipe and 1F1B. Figure 1 displays the effects of different pipelining schedules, with and without the use of 2BP.

3.2 Implementation

Our implementation is built on top of PyTorch. However, we do not use PyTorch's automatic differentiation engine (`torch.autograd`). This granted us greater flexibility when implementing 2BP. We replaced `torch.autograd` by manually implementing the backward pass operation for all of the modules employed within the models we chose to benchmark. Each module has a forward and a backward-p1 function; if that module contains parameters (equivalent to `torch.nn.Parameter`) then it also has a backward-p2 function. We can simulate the behaviour of `torch.autograd` by calling backward-p2 directly after a backward-p1 call as we backward propagate through the compute graph, instead of delaying the computation.

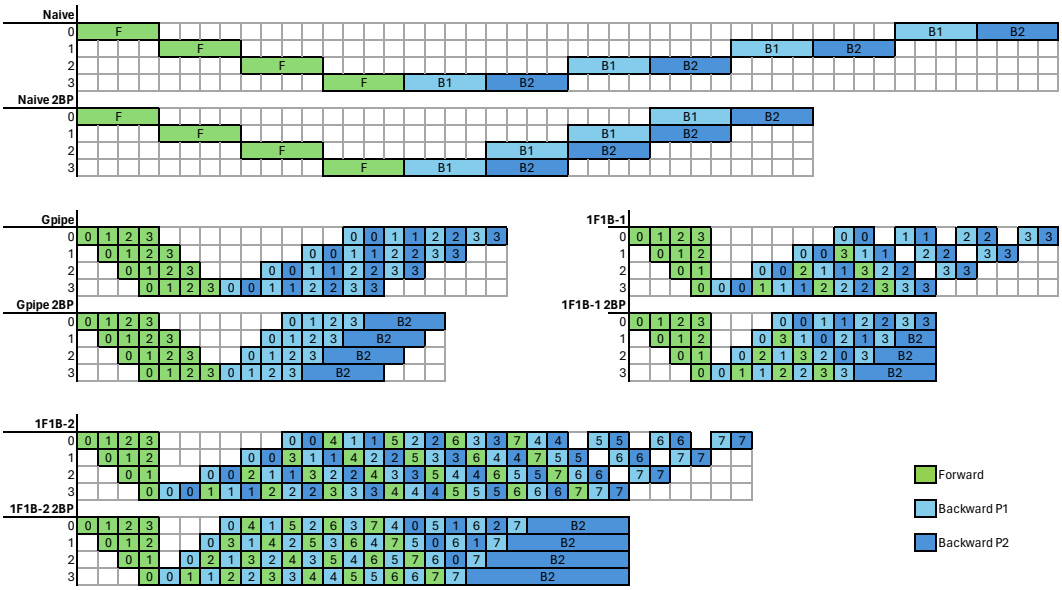

Figure 1: Pipelining schedules (Naive, Gpipe, 1F1B-1, 1F1B-2) with and without 2BP. This figure assumes that the time taken to compute the forward, backward-p1 and backward-p2 passes are equal.

We test the effects of 2BP on 4 model architectures across 4 pipelining schedules. The first model chosen was a transformer architecture [33] with 7 billion parameters (Transformer-7b). This model is based on the LLaMa [32] and PaLM [3] architectures (steps including rotary embedding [30], SwiGLU MLP [28], RMSNorm [36], no linear bias). Transformer-7b has a context length of 1024

and a model dimension of 4096. The second chosen model is another well-known bidirectional transformer model, BERT-Large [5]. To expand the test set beyond transformer models, a relatively new architecture, Mamba-1.4B [10] was chosen. Lastly, to evaluate the performance on an architecture with a non-uniform computational graph, we have chosen ResNet-152 [12], a convolutional neural network (CNN) model. A model with a non-uniform compute graph refers to a model who's activations do not share a constant shape throughout the model, transformers are an example of a model with a uniform compute graph as their activations keep a constant shape at every block. By selecting these four diverse models, we ensure a comprehensive assessment of 2BP across various architectural designs and computational demands.

The 4 pipelining schedules we test are Gpipe, 1F1B-1, 1F1B-2 and a naive approach with no scheduling algorithm and maximum bubble time. 1F1B is split into 2 categories: 1F1B-1 which indicates that the mini-batch is split into N micro-batches where N is the number of pipeline processes (also equal to the number of accelerators), and 1F1B-2 which indicates that the mini-batch is split into $2N$ micro-batches.

For our GPipe implementation, we delay the computation of backward-p2 until all micro-batches have finished both forward and backward-p1 steps. We then concatenate the activations and intermediate derivatives of all micro-batches over the batch dimensions, meaning we only have to call backward-p2 once rather than N times as shown in Figure 2. For both 1F1B implementations (without 2BP), every accelerator (except the last) has some idle time after their backward-p1 calls. After the backward-p1 operation has been called for a micro-batch, all the necessary intermediates have been calculated and stored in memory for use in the computation of backward-p2 for that micro-batch. This allows us to fill that idle time between backward-p1 calls with backward-p2 calls (see Figure 1 for a visual representation). Once all the backward-p1 steps have been called, we concatenate the remaining forward and backward-p1 intermediates over the batch dimension and compute backward-p2 the same way as was done with GPipe. When working with models with non-uniform compute graphs (such as CNNs), executing backward-p2 steps during idle time may not be optimal, since the backward-p2 step may take longer than the original idle time, resulting in excess idle time on a different accelerator.

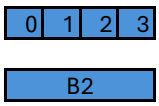

Figure 2: Combining each microbatch's backward-p2 into a single operation.

Table 1 shows the theoretical bubble ratios as functions of N where N is the number of pipeline processes. Bubble ratio is defined as the fraction of total runtime which is idle, i.e. idle time divided by the total runtime. These functions assume that the forward, backward-p1 and backward-p2 functions take equal lengths of time, which does not hold true in practice but acts as a good estimate for comparing the performance scaling for the different schedules as the number of processes varies. The throughput gain is the speedup of the total runtime from using and not using 2BP, and can be obtained from $\frac{1-b}{1-a}$, where b is the bubble ratio with 2BP and a is the bubble ratio without.

Table 1: Bubble ratios and throughput gains for each pipelining schedule. Equal time for forward, backward-p1, and backward-p2 assumed.

Pipelining Schedule	Bubble Ratio	2BP Bubble Ratio	Throughput Gain
Naive	$\frac{N-1}{N}$	$\frac{2(N-1)}{2N+1}$	$\frac{3N}{2N+1}$
GPipe	$\frac{N-1}{2N-1}$	$\frac{2(N-1)}{2(N-1)+3N}$	$\frac{3(2N-1)}{2(N-1)+3N}$
1F1B-1	$\frac{N-1}{2N-1}$	$\frac{N-1}{N-1+3N}$	$\frac{3(2N-1)}{N-1+3N}$
1F1B-2	$\frac{N-1}{3N-1}$	$\frac{N-1}{N-1+6N}$	$\frac{3(3N-1)}{N-1+6N}$

We also used `torch.jit.script` to compile the backward-p1 operations for both softmax and RMSNorm, resulting in significant speed up for each. We do not include this pre-training compilation

step in our timing measurements, since this is required whether 2BP is in place or not. The data collected came from training on randomly generated data. This is done instead of using an actual dataset since from our experience with the system, dataloading can be a significant bottleneck and optimising dataloading is beyond the scope of this paper. All of the throughput results were collected from training on randomly generated data samples of size 16384 and averaged over 4 epochs.

4 Evaluation

Our experiments were performed on two systems at EPCC: the Edinburgh International Data Facility (EIDF) GPU-Service [6, 22] and Cirrus [4]. Most of the results were collected from the GPU nodes on the EIDF GPU-service containing 4 40GB Nvidia A100 connected via SXM4 interconnect, and the Mamba throughput results were collected on nodes with 4 80GB Nvidia A100s connected via SXM4 interconnect. Cirrus supports multi-node GPU jobs, which is particularly useful for scaling tests. Cirrus GPU nodes contains 4 16GB Nvidia V100 GPUs connected via an SXM2 interconnect. For our PyTorch implementation, since autograd is not used, we are able to perform our runs with PyTorch's inference mode activated. PyTorch's distributed library is used for p2p communication with a NCCL [31] backend.

Table 2 summarises the model architecture used for our experiments. Although the optimizer [16, 20] chosen for each benchmark is not relevant for this paper, the optimizer calculations are taken into account during the throughput measurements. The naive implementation does not use micro-batches but uses a mini-batch of $4\times$micro-batch size. When training ResNet152, in order to keep the batch normalization [15] computation equal between pipeline schedule, the naive implementation uses a mini-batch size of 8 and 4 gradient accumulation steps. All models with the exception of ResNet152 distributed the number of blocks equally amongst the 4 GPUs (excluding the embedding blocks and prediction heads where appropriate). ResNet152 contains 50 ResNet bottlenecks which were split in the ratio [10, 14, 14, 12] across the 4 GPUs respectively. ResNet152 also contains some initial convolutions computed by GPU 0 and a classification head computed by GPU 3. The loss is always handled by GPU 3 as it computes the model's output.

Table 2: Model hyperparameters used for benchmarking.

Model	Data type	Micro-Batch size	Optimizer
Mamba-1.4b	fp16	2	AdamW
LLaMa-7b	fp16	1	Adam
ResNet152	fp32	8	SGD
BERT-Large	fp16	2	Adam

4.1 2BP throughput

Figure 3 summarises the throughput results for different models with different pipeline schedules, with and without 2BP. Speedup can be observed for every case with the use of 2BP, ranging from **1.10x** for 1F1B-1 on ResNet152 to up to **1.70x** for 1F1B-1 on Transformer-7b. The observed difference in performance gain for different models may be caused by multiple factors, including the non-uniformity of the ResNet152 model results in unwanted idle compute time, as well as the smaller problem size compared to models like Transformer-7b, this would also explain the smaller performance gain of BERT-Large at **1.26x**.

The time taken for the backward-p1 step compared to backward-p2 step is usually uneven, and depends heavily on the model architecture. For example, for 2D batch normalization, the backward-p2 operation is significantly simpler than the backward-p1 operation. Furthermore, some operations, e.g. the scalar dot-product attention, do not require a backward-p2 operation but have a significant backward-p1 operation. This difference in computational complexity between backward-p1 and backward-p2 may cause the variation of performance gain observed between model architectures.

4.2 2BP memory consumption

While 2BP is able to reduce idle computation time, it comes at the cost an increase in peak memory usage. Figure 4 summarises the effect on peak GPU memory usage (i.e. the GPU which reserved the

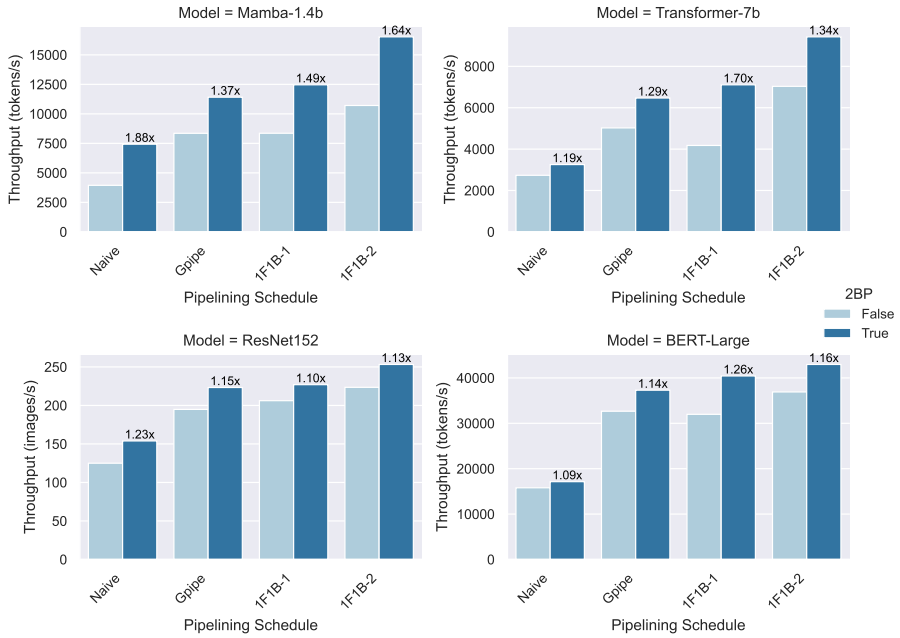

Figure 3: Sample throughput for each model with different pipeline schedules. Light blue bars represent schedule without 2BP, and dark blue represents with 2BP. Numbers above bars represent throughtput gain from using 2BP.

maximum memory) for each model and pipeline schedule, with and without 2BP. It can be seen that the increase in memory consumption varies significantly between the model architectures but also the pipeline schedules. In Mamba with the 1F1B-2 pipelining schedule, we see an increase in memory of **2.67x**; on the other hand, for training Transformer-7b with the 1F1B-1 pipelining schedule, the memory consumption only increases by **1.02x**.

The increase in memory is a result of 2 different characteristics of 2BP: first the intermediate derivatives $\frac{\partial L}{\partial z_l}$ have to be stored in memory after backward-p1 for reuse in backward-p2. Second, the activations that are needed for the delayed backward-p2 calls are held for longer. The GPU with the largest memory consumption is dependent on the pipelining schedule being used and the uniformity of the models compute graph.

The specific operations used in a model can result in a significant increase in peak memory; for Linear and Convolution layers, both the input activations and output derivatives need to be stored in memory for backward-p2. On the other hand, operations that are purely functional such as ReLU and Scalar Dot-Product Attention release their activations during the backward-p1 calls.

Comparing the memory footprint for different pipelining schedules, Gpipe requires all process to save the activations of all the micro-batches for the backward pass, resulting in a large memory footprint for each device. For 1F1B-1, without the use of 2BP, GPU 0 will always have the largest activation memory since it has to store the activations for N micro-batches; with 2BP, the GPU with the greatest activation memory is now dependent on the model architecture. Although GPU 0 has the activations of all micro-batches saved in memory, it only ever has to store 1 micro-batch worth of intermediate derivatives. GPU $N-1$ has to store N micro-batches worth of intermediate derivatives. The peak memory required by activations is dependent on how many activations are released after backward-p1.

The reason for the large increase in peak memory usage for 1F1B-2 as seen by the is the fact that the majority of the activations and intermediate derivatives need to be held in memory until backward-p2 is called. Although we did not implement this, by calling backward-p2 on some of the micro-batches halfway through the 1F1B-2 training step, as visualised in Figure 5, it may be possible to recover the peak memory usage closer to that of 1F1B-1. As further work to this study, we would like to investigate how the frequency at which backward-p2 is called during a training

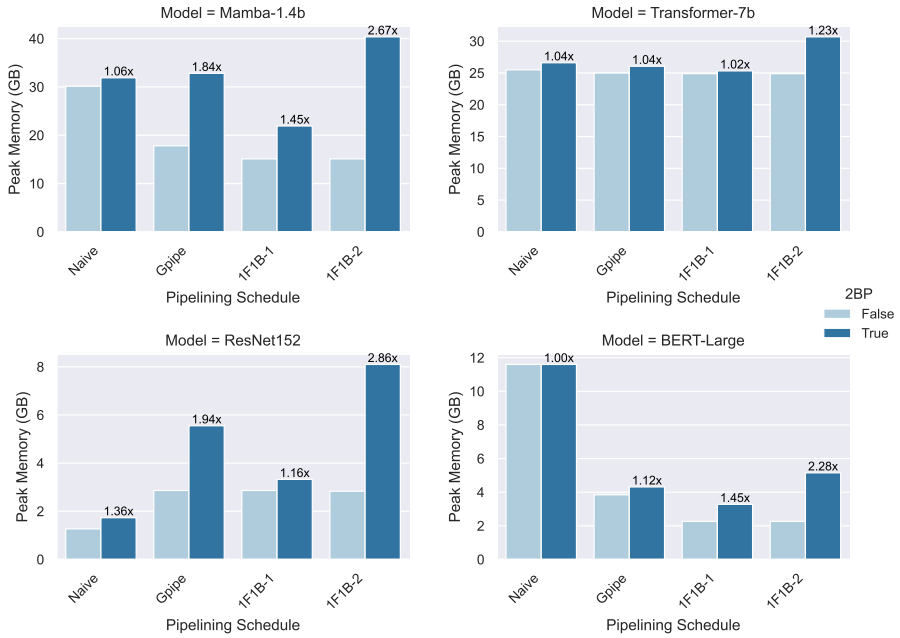

Figure 4: Maximum memory usage across the 4 GPUs. The "peak memory (GB)" is measured by obtaining the peak reserved memory on each GPU and taking then the maximum. Light blue bars represent schedule without 2BP, and dark blue represents with 2BP. Numbers above bars represent the increase in memory from using 2BP.

step affects performance and memory consumption, especially as we scale to versions of 1F1B with greater numbers of micro-batches e.g. $8N$ micro-batches.

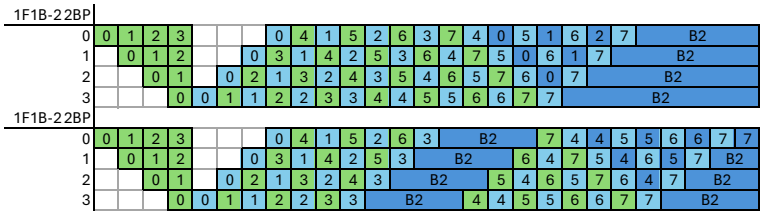

Figure 5: Alternative memory efficient schedule for 1F1B-2 with 2BP

4.3 Scaling

In this section we look into the effects of scaling on 2BP. Table 1 shows that as the number of accelerators N increases, the bubble ratio increases. At the same time the performance gain between training with and without 2BP should increase since the rate at which the bubble ratio increases is greater when not using 2BP. Using a BERT-like model, we test the effect of scaling when using 1F1B-1 and 1F1B-2 as they give the greatest throughput. Both implementations use a micro-batch size of 2.

4.3.1 Fixed model size

We use a BERT-like model with 32 blocks to perform a scaling test with fixed model size. Figure 6 summarises the effect of 2BP with varying number of accelerators. For 1F1B-1, the performance increase from 2BP goes from **1.21x** for 4 GPUs, to **1.20x** for 8 GPUs, and **1.18x** for 16 GPUs. For 1F1B-2, the gain is **1.15x**, **1.14x** and **1.11x** respectively. Even though the theoretical prediction from Table 1 suggests that the throughput gain should increase as the number of accelerators increase, the

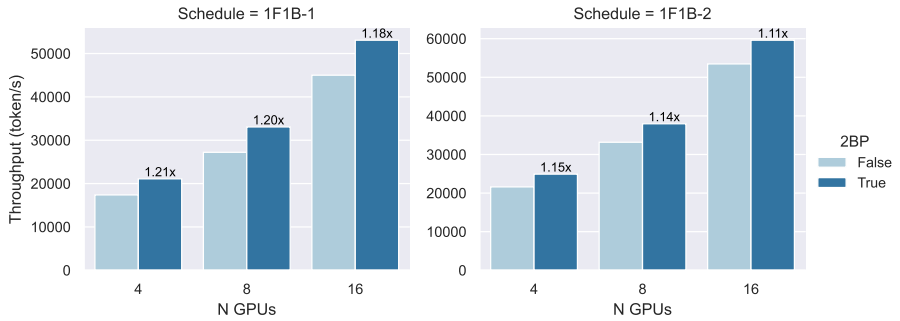

Figure 6: Scaling the number of GPUs with a fixed model size. Light blue bars represent schedule without 2BP, and dark blue represents with 2BP. Numbers above bars represent throughtput gain from using 2BP.

formula did not take into account the increase in communication required, especially when going above 4 GPUs (on our system) requires inter-node communication.

4.3.2 Variable model size

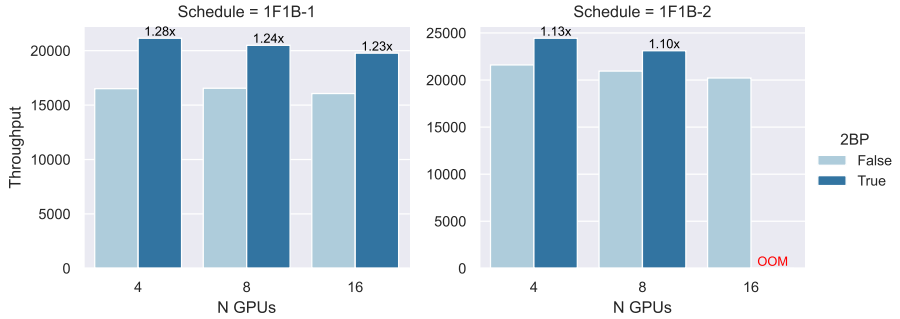

Figure 7: Scaling the number of GPUs with a variable global model size. Light blue bars represent schedule without 2BP, and dark blue represents with 2BP. Numbers above bars represent throughtput gain from using 2BP.

Here we test the performance as we scale the model size with the number of processors; each accelerator computes 8 BERT-like blocks. From Figure 7, we see that for 1F1B-1, the performance gains are **1.28x**, **1.24x**, and **1.23x** for 4, 8, and 16 GPUs. For 1F1B-2 they are **1.13x** and **1.10x** for 4 and 8 GPUs, whereas the 16 GPU run resulted in an out of memory error. This error is caused by storing the activations and intermediate derivatives of 16 micro-batches on GPU $N-1$. As stated in the previous section, the performance gain should in theory increase with the number of GPUs; however we observe a similar degradation in performance gain, which is most likely due to the increase in communication required.

4.4 GPU compute occupancy

As mentioned in section 3, micro-batches are concatenated (copied into contiguous memory location) across their batch dimensions during the compute of backward-p2. In theory this should allow for a greater utilization of compute resources when working with GPUs due to their SIMT architecture. We tested the effect of this concatenation, and the results are summarised in Table 3. In practice, we did not observe a significant difference in whether the concatenation was performed, or if the N backward-p2 steps were simply performed in a loop (see Figure 2). It is possible that concatenation does provide speed up, but the concatenation step itself is time consuming and neutralises the benefits.

As the necessary inputs for all backward-p2 operations across the entire model exist in memory at the time of calling the first backward-p2 operation, in theory all backward-p2 operations can be called in

parallel. The easiest way to perform this on Nvidia GPUs is via CUDA streams. PyTorch provides an API to use CUDA streams, but we observed that it drastically increased the compute time of the backward-p2 operations, to the point at which it was faster to call each operation in serial. CUDA graphs may be a alternative method to achieve further parallelisation during backward-p2 calls, which we intend to explore in the future.

Table 3: Average throughput using the 1F1B-1 pipelining schedule and 2BP, with and without concatenating micro-batches during the backward-p2 step.

Model	Avg Throughput w/	Avg Throughput w/o
Transformer-7b	7120.88	7100.69
Bert-Large	40427.41	40387.41
Mamba-1.4b	12437.91	12431.13
ResNet152	194.93	193.10

5 Further work

As stated previously, 2BP can cause an increase in activation memory. Here we propose potential methods to decrease the effects of the increase in activation memory, which we will investigate in the future. Firstly, by performing intermediate derivative checkpointing, which would work similarly to activation checkpointing [21] in which some activations are not stored in memory but recomputed during the backward pass, but applied to the intermediate derivates. Activation checkpointing is a widely used practice for training DNNs. The intermediate derivative recalculations could potentially be overlapped with idle compute to result in minimal performance decreases compared to running 2BP without intermediate derivative checkpointing.

Another potential way to decrease memory consumption caused by storing the intermediate derivatives would be to temporarily store the intermediates from the first few micro-batches into either host memory or NVMe storage [24, 27], and the number of micro-batches to offload would depend on pipelining schedule and GPU rank.

When training large DNNs, pipeline parallelism is usually not the sole distributed parallelism paradigm employed. Data parallelism is very often used along side pipeline parallelism, and data parallelism can be optimised by overlapping communication of the gradients with computation during the backward propagation through the network. As the gradients are not calculated until the delayed computation during backward-p2, we expect it will be significantly harder to fully overlap communication with computation, especially when working on systems with slower communication infrastructure. We aim to explore this in the future.

6 Conclusion and discussion

For this work, we have applied 2BP, which splits the backward propagation step into 2 stages, to training 4 different DNN model architectures. We have demonstrated that this is able to reduce the idle compute time, resulting in significant increase in throughput on top of the SOTA pipelining schedules.

As part of our implementation of 2BP, the backward pass operations had to be manually implemented; these operations have been implemented by PyTorch but are not exposed to be used functionally though PyTorch's Python API. The authors would like to see this functionality exposed to users, allowing for easier and more granular control over the implementation of custom backward propagation methods such as the one demonstrated in this work.

Acknowledgments and Disclosure of Funding

This work used the Cirrus UK National Tier-2 HPC Service at EPCC (http://www.cirrus.ac.uk) funded by the University of Edinburgh and EPSRC (EP/P020267/1). This work was supported by the Edinburgh International Data Facility (EIDF) and the Data-Driven Innovation Programme at

the University of Edinburgh. For the purpose of open access, the author has applied a Creative Commons Attribution (CC BY) licence to any Author Accepted Manuscript version arising from this submission.

References

[1] Martín Abadi et al. *TensorFlow, Large-scale machine learning on heterogeneous systems*. Nov. 2015. DOI: 10.5281/zenodo.4724125.

[2] Jason Ansel et al. "PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation". In: *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. ACM, Apr. 2024. DOI: 10.1145/3620665.3640366. URL: https://pytorch.org/assets/pytorch2-2.pdf.

[3] Aakanksha Chowdhery et al. *PaLM: Scaling Language Modeling with Pathways*. 2022. arXiv: 2204.02311 [cs.CL].

[4] *Cirrus Hardware — cirrus.ac.uk*. https://www.cirrus.ac.uk/about/hardware.html. [Accessed 26-03-2024]. Mar. 2022.

[5] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. arXiv: 1810.04805 [cs.CL].

[6] *Edinburgh International Data Facility — edinburgh-international-data-facility.ed.ac.uk*. https://edinburgh-international-data-facility.ed.ac.uk/. [Accessed 22-03-2024]. Mar. 2022.

[7] FairScale authors. *FairScale: A general purpose modular PyTorch library for high performance and large scale training*. https://github.com/facebookresearch/fairscale. 2021.

[8] Shiqing Fan et al. *DAPPLE: A Pipelined Data Parallel Approach for Training Large Models*. 2020. arXiv: 2007.01045 [cs.DC].

[9] Jiarui Fang et al. "Parallel Training of Pre-Trained Models via Chunk-Based Dynamic Memory Management". In: *IEEE Transactions on Parallel and Distributed Systems* 34.1 (Jan. 2023), pp. 304–315. ISSN: 2161-9883. DOI: 10.1109/tpds.2022.3219819. URL: http://dx.doi.org/10.1109/TPDS.2022.3219819.

[10] Albert Gu and Tri Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. 2023. arXiv: 2312.00752 [cs.LG].

[11] Aaron Harlap et al. *PipeDream: Fast and Efficient Pipeline Parallel DNN Training*. 2018. arXiv: 1806.03377 [cs.DC].

[12] Kaiming He et al. *Deep Residual Learning for Image Recognition*. 2015. arXiv: 1512.03385 [cs.CV].

[13] Haichen Huang et al. *Elixir: Train a Large Language Model on a Small GPU Cluster*. 2023. arXiv: 2212.05339 [cs.DC].

[14] Yanping Huang et al. *GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism*. 2019. arXiv: 1811.06965 [cs.CV].

[15] Sergey Ioffe and Christian Szegedy. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. 2015. arXiv: 1502.03167 [cs.LG].

[16] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980 [cs.LG].

[17] Shenggui Li et al. *Colossal-AI: A Unified Deep Learning System For Large-Scale Parallel Training*. 2023. arXiv: 2110.14883 [cs.LG].

[18] Shigang Li and Torsten Hoefler. "Chimera: efficiently training large-scale neural networks with bidirectional pipelines". In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. SC '21. ACM, Nov. 2021. DOI: 10.1145/3458817.3476145. URL: http://dx.doi.org/10.1145/3458817.3476145.

[19] Ziming Liu et al. "Hanayo: Harnessing Wave-like Pipeline Parallelism for Enhanced Large Model Training Efficiency". In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. SC '23. ACM, Nov. 2023. DOI: 10.1145/3581784.3607073. URL: http://dx.doi.org/10.1145/3581784.3607073.

[20] Ilya Loshchilov and Frank Hutter. *Decoupled Weight Decay Regularization*. 2019. arXiv: 1711.05101 [cs.LG].

[21] E. Naveen and P. Kumar. "Checkpointing in Practice for Memory-Efficient Training on the Edge". In: *2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 17th International Conference on Smart City; IEEE 5th International Conference on Data Science and Systems (HPCC/SmartCity/DSS)*. 2019, pp. 2759–2766. DOI: 10.1109/HPCC/SmartCity/DSS.2019.00387.

[22] *Overview - EIDF User Documentation — epcced.github.io*. https://epcced.github.io/eidf-docs/services/gpuservice/. [Accessed 26-03-2024]. Mar. 2022.

[23] Penghui Qi et al. *Zero Bubble Pipeline Parallelism*. 2023. arXiv: 2401.10241 [cs.DC].

[24] Samyam Rajbhandari et al. *ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning*. 2021. arXiv: 2104.07857 [cs.DC].

[25] Samyam Rajbhandari et al. *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*. 2020. arXiv: 1910.02054 [cs.LG].

[26] Jeff Rasley et al. "DeepSpeed: System Optimizations Enable Training Deep Learning Models with Over 100 Billion Parameters". In: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD '20. Virtual Event, CA, USA: Association for Computing Machinery, 2020, pp. 3505–3506. ISBN: 9781450379984. DOI: 10.1145/3394486.3406703. URL: https://doi.org/10.1145/3394486.3406703.

[27] Jie Ren et al. *ZeRO-Offload: Democratizing Billion-Scale Model Training*. 2021. arXiv: 2101.06840 [cs.DC].

[28] Noam Shazeer. *GLU Variants Improve Transformer*. 2020. arXiv: 2002.05202 [cs.LG].

[29] Mohammad Shoeybi et al. *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism*. 2020. arXiv: 1909.08053 [cs.CL].

[30] Jianlin Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. 2023. arXiv: 2104.09864 [cs.CL].

[31] *The NVIDIA Collective Communication Library (NCCL)*. https://github.com/NVIDIA/nccl. [Accessed 27-03-2024]. Mar. 2022.

[32] Hugo Touvron et al. *LLaMA: Open and Efficient Foundation Language Models*. 2023. arXiv: 2302.13971 [cs.CL].

[33] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL].

[34] Bowen Yang et al. *PipeMare: Asynchronous Pipeline Parallel DNN Training*. 2020. arXiv: 1910.05124 [cs.DC].

[35] PengCheng Yang et al. "Group-based Interleaved Pipeline Parallelism for Large-scale DNN Training". In: *International Conference on Learning Representations*. 2022. URL: https://openreview.net/forum?id=cw-EmNq5zfD.

[36] Biao Zhang and Rico Sennrich. *Root Mean Square Layer Normalization*. 2019. arXiv: 1910.07467 [cs.LG].